

- The assignment is due at Gradescope on 3/31/2025.
- A LaTeX template will be provided for each homework. You are strongly encouraged to type your homework into this template using $\text{\LaTeX}$. If you are writing by hand, please fill in the solutions in this template, inserting additional sheets as necessary. This will help facilitate the grading.
- You are permitted to discuss the problems with up to 2 other students in the class (per problem); however, *you must write up your own solutions, in your own words*. Do not submit anything you cannot explain. If you do collaborate with any of the other students on any problem, please list all your collaborators in the appropriate spaces.
- Similarly, please list any other source you have used for each problem, including other textbooks or websites.
- *Show your work*. Answers without justification will be given little credit.
- Your homework is *resubmittable*. Please refer to the course syllabus on Canvas for a more detailed description of this. For any problem that you have not changed from your last submission, please make sure to indicate this in your submission to help our graders grade faster.

PROBLEM 1 (SYLLABUS READING COMPREHENSION) *The following questions all pertain to course policy. None are trick questions, and all can be found in the syllabus on Canvas, or in the policies on the first page of this assignment. For this question, please enter your answer directly as text in Gradescope under HW1 - Problem 1.*

- (a) You're having a tough first week back, and you want to turn in this homework assignment four days late. What point penalty will you incur on your submission? What benefits exist for turning your assignment in on time?
- (b) You got an N on problem 3, but an E on everything else. You read the feedback, went to OH, and have a better solution now. You want to resubmit it, but you've made no changes to any of your other solutions. How should you format your solution for resubmission in Gradescope?

Collaborators:

Solution: Your solution goes here.

Extra space for your solution

PROBLEM 2 (PRACTICE WITH BIG O.) For this problem, you may want to review the definition of Big O, Big Ω and Big Θ [see DPV 0.3]. Helpful resources can also be found on Canvas. In each of the following situations, indicate whether $f = O(g)$, or $f = \Omega(g)$, or both (in which case $f = \Theta(g)$), and **give a brief explanation** for each answer.

(a) $f(n) = \sum_{i=1}^n i^k$ and $g(n) = n^{k+1}$ for constant $k > 0$.

(b) $f(n) = \sum_{i=1}^n i^k$ and $g(n) = n^{k+1}$ for $k \in \Theta(\log n)$.

(c) $f(n) = \binom{n}{5}$ and $g(n) = \sqrt{n}^{3 \log_2 8}$

(d) $f(n) = 3^n$ and $g(n) = \sum_{i=1}^n 2^i$

(e) $f(n) = \log(n!)$ and $g(n) = n \log n$

(f) $f(m, n) = (n + m)^2$ and $g(m, n) = n^2 + m^2$

Collaborators:

Solution: Your solution goes here.

Extra space for your solution

PROBLEM 3 (“CONFLICTING” DEFINITIONS) Surprisingly, our DPV textbook and the KT textbook provide different definitions for big-O notation. For $f, g : \mathbb{N}_{>0} \rightarrow \mathbb{R}_{>0}$, they are:

- DPV: $f(n) = O(g(n)) \iff \exists c \in \mathbb{R}, \forall n \in \mathbb{N}_{>0}, f(n) \leq c \cdot g(n)$
- KT: $f(n) = O(g(n)) \iff \exists c \in \mathbb{R}, \exists n_0 \in \mathbb{N}_{>0}, \forall n \geq n_0, f(n) \leq c \cdot g(n)$.

Write a **formal proof** that the two definitions are equivalent.

Extra space for your solution

PROBLEM 4 (FASTER FIBONACCI) This exercise is similar to 0.4 from [DPV]. Recall that the Fibonacci sequence is defined recursively by:

$$F_0 = 1, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2}.$$

Is there a faster way to compute the n^{th} Fibonacci number than by `fib2` (DPV, pg. 13)? One idea involves matrices.

If you are not already familiar with matrix multiplication, you will need to learn just a bit of it here. For this problem, you should familiarize yourself with the following:

- [How to convert a linear system to a matrix equation](#)
- [How to multiply matrices](#)
- Know that matrix multiplication is associative.

We start by writing the equations $F_1 = F_1$ and $F_2 = F_0 + F_1$ in matrix notation:

$$\begin{pmatrix} F_1 \\ F_2 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} F_0 \\ F_1 \end{pmatrix}$$

Similarly:

$$\begin{pmatrix} F_2 \\ F_3 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} F_1 \\ F_2 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^2 \cdot \begin{pmatrix} F_0 \\ F_1 \end{pmatrix}$$

and in general:

$$\begin{pmatrix} F_n \\ F_{n+1} \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^n \cdot \begin{pmatrix} F_0 \\ F_1 \end{pmatrix}.$$

So, in order to compute F_n , it suffices to raise this 2×2 matrix, call it X , to the n^{th} power.

- Show that two 2×2 matrices can be multiplied using 4 additions and 8 multiplications.
- Give an algorithm to compute X^n using only $O(\log n)$ matrix multiplications. You should provide pseudocode for your algorithm, and give a **formal proof** that it works correctly and only uses $O(\log n)$ matrix multiplications (Hint: Think about computing X^8).
- We name the algorithm from part (b) `fib3`. **Sketch** a proof that all intermediate results of `fib3` are $O(n)$ bits long.
- Let $M(n)$ be the running time of an algorithm for multiplying n -bit numbers, and assume that $M(n) = O(n^2)$ (the school method for multiplication, in Chapter 1 of DPV, achieves this). **Sketch** a proof that the running time of `fib3` is $O(M(n)\log n)$.
- Sketch** a proof that the running time of `fib3` is $O(M(n))$. (Hint: The lengths of the numbers being multiplied get doubled with every squaring.)

Collaborators:

Solution: Your solution goes here.

You may follow this template when writing pseudocode:

Algorithm 1 Pseudocode Example.

Input: Two squared matrices B and C , and a number i

Output: A squared matrix A

```
1: // Some example code: an IF statement
2: if  $i > 1$  then
3:     // Some example code: Multiplying two matrices
4:      $A \leftarrow B * C$ 
5: else
6:      $A \leftarrow C * B$ 
7: end if
8: Return  $A$ 
```

Extra space for your solution

PROBLEM 5 (FIBONACCI IMPLEMENTATION) *For this problem, you will be asked to implement a function that computes the value of the n^{th} Fibonacci number, and compare its runtime to that of the algorithms seen in class. You can find the questions [here](#). Create a copy of the document, implement the required code, download the Jupyter Notebook, and submit it to the appropriate Gradescope Assignment.*

Collaborators:

Solution: Your solution's link goes here.